

Project Report: Sentiment Analysis on Amazon Product Reviews

1. Dataset Overview

We used the Amazon product review dataset containing ~20,000 reviews.

- **reviewText** → Textual content of the product review.
 - **Positive** → Binary sentiment label (1 = positive, 0 = negative).
-

2. Data Preprocessing

- Checked for missing values — none were found.
 - Added `review_length` and `word_count` as additional features.
 - Cleaned text by:
 - Lowercasing.
 - Removing HTML tags, punctuation, numbers, and stopwords.
 - Applied lemmatization for word normalization.
 - Vectorized text using:
 - **TF-IDF** (for ML models).
 - **Tokenization + Padding** (for DL models).
 - Addressed class imbalance using **SMOTE (Synthetic Minority Over-sampling Technique)** on training data to balance positive and negative classes.
-

3. Exploratory Data Analysis (EDA)

- Plotted class distribution → ~76% positive, ~24% negative.
 - Visualized review lengths and word counts.
 - Displayed top 20 most common words (after removing stopwords).
 - Checked and confirmed no duplicate reviews.
-

4. Model Selection and Training

We trained two groups of models:

Machine Learning models:

- Logistic Regression
- Random Forest
- Support Vector Machine (SVM)
- Naive Bayes
- XGBoost
- Voting Classifier Ensemble (combining LR, RF, SVM)

Deep Learning models:

- LSTM
- Bidirectional LSTM
- Bidirectional GRU

Deep Learning details:

- Used embedding layers.
- Applied bidirectional RNNs.

- Added early stopping to prevent overfitting.
-

5. Evaluation Metrics

We evaluated models using:

- Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion Matrix
 - ROC Curve (for ML models)
-

6. Hyperparameter Tuning

- Tuned key hyperparameters:
 - `C` (Logistic Regression, SVM)
 - `n_estimators` (Random Forest)
 - `max_depth` (Random Forest)
 - Applied **Keras Tuner** for LSTM:
 - Tuned embedding dimension, LSTM units, dropout rate, and learning rate.
-

7. Comparative Analysis

Model	Accuracy	Notes
Logistic Regression	~89%	Strong baseline, good balance
Random Forest	~86%	Slightly weaker on minority class
SVM	~90%	Best ML performance
Naive Bayes	~85%	Fast but struggled on class 0
XGBoost	~87%	Robust, well-balanced
Voting Ensemble	~90%	Combines best ML models, strong overall
Bidirectional LSTM	~90%	Best DL model, stable performance
Bidirectional GRU	~89%	Comparable to BiLSTM, slightly slower
Tuned LSTM	~90%	Improved precision/recall after tuning

8. New Ensemble & Tuned DL Results

- **Voting Classifier Ensemble (after GridSearchCV tuning):**

- Accuracy: 89.7%
- Class 0: Precision 0.86, Recall 0.68
- Class 1: Precision 0.91, Recall 0.96

Confusion Matrix:

```
[[ 650  303]
 [ 108 2939]]
```

○

- **Tuned LSTM (after Keras Tuner tuning):**

- Accuracy: 90%
- Class 0: Precision 0.79, Recall 0.76

- Class 1: Precision 0.93, Recall 0.94

Confusion Matrix:

```
[[ 729  224]
 [ 188 2859]]
```

-

9. Conclusion

- **Best-performing models:**
 - ML → SVM and Voting Classifier Ensemble
 - DL → Tuned Bidirectional LSTM
 - **Key findings:**
 - SMOTE significantly improved class balance and boosted performance.
 - Hyperparameter tuning raised both ML and DL performance.
 - Bidirectional RNNs enhanced deep learning results.
 - ML models remained surprisingly competitive against DL models.
-

10. Challenges and Lessons Learned

- Addressing class imbalance with SMOTE was essential.
- Deep learning requires long training time and GPU resources.
- Traditional ML models provided strong performance on text data.
- Careful preprocessing (cleaning + lemmatization) was critical.
- Hyperparameter tuning made a clear performance difference.

